Hes·so

haute école
neuchâtel berne jura
arc ingénierie
www.he-arc.ch

Bachelor in Computer Science and Communication Systems

Study Field : Software Engineering

P3 - Linux Syscall Interception Server

Réalisé par :
Catalin BOZAN

Sous la direction de :
Prof. Marcelo Pasin
Haute Ecole Arc, HES-SO

January 23, 2026

Abstract

The purpose of this project is to create a proof of concept server that receives Linux syscall requests from a client, executes them, and sends back their response.

This has applications in confidential computing environments[1] where a target program is running in a 'sealed' virtual environment, but still needs some access to the host's Linux kernel.

The target program's syscalls are intercepted by a client that forwards the syscalls to the server that is running somewhere else, for example on the host machine. The server executes the syscalls and returns their response to the target program, as if the syscall was performed in the target program's environment.

Contents

Chapter 1

Project Plan

In order to create a syscall server, there must also be a process and a client that intercepts this process's syscalls and forwards them to the server.
Let's begin by first having a look at the normal execution of a process in a Linux-based operating system: the process communicates directly with the kernel.

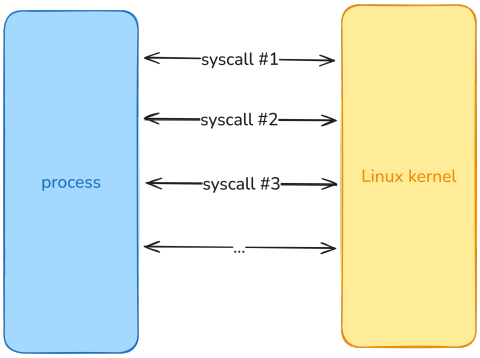

Figure 1.1: Direct syscalls to Linux kernel

The goal of this project is to create a server capable of receiving and resolving syscall requests as shown in the diagram below.

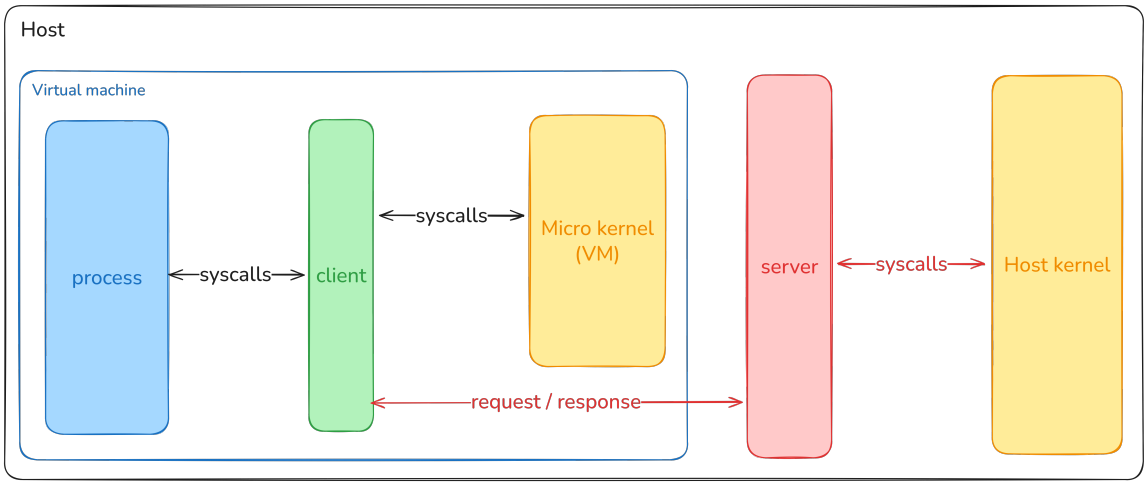

Figure 1.2: Syscall interception and forwarding in a containerized environment

Although this project's focus is the server, we also need a target program and client in order to test the server's functionality.
For simplicity's sake, during the development of the server, the target program and client will run directly on the host machine. As shown in the diagram below.

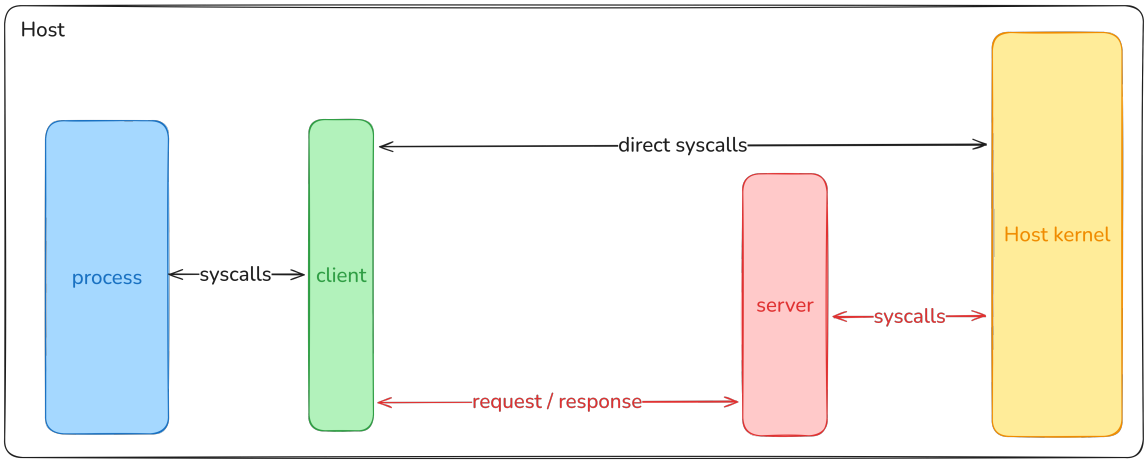

Figure 1.3: Syscall interception and forwarding in the context of this project's development

As you can see, not all of the syscalls are intercepted by the client. This is because some operations must be run directly by the target process in order to function properly, for example memory allocation.

Chapter 2

Research

There are many topics to be research and decisions to be made in order to start this project. In this chapter I will go over them one by one.

2.1 Programming language

Since we are working with Linux system calls, I picked the obvious choice for the programming language to be used in this project: C.
This is because there is a native Linux syscall api for the C programming language, since the kernel itself is written in C.

2.2 Syscall interception

There were two approaches I considered when it comes to system call interception:

- Modifying the standard C system library with my own syscall implementations.
- Creating a shared library containing custom syscall implementations, and loading it before before all the other shared libraries.

I chose the latter, since it's easier to implement and it can be used with any program as long as it's not statically compiled, which is the case for most programs.

2.2.1 Which syscalls to implement

There are hundreds of system calls that the Linux kernel supports. Because of the relatively limited time allocated for this project, I chose to implement some of the most common ones.

The first system calls that came to my mind were file operations like open(), read(), write() and close(). This was a good start, but in order to have a clearer idea of what system calls to implement, I have decided to take an existing program and have a look at what system calls it uses.

The program I chose to analyze is sqlite3, since it's a widely used tool that isn't too complex in terms of what system operations it needs.

I started by using the strace tool, in order to see what system calls are used by sqlite3. I soon found out that sqlite3 heavily relies on one particular system call: mmap().
This syscall allows a program to map files to a certain region in memory, so that all changes made to that specific memory region are reflected onto the file.
Since there is no practical way of intercepting direct memory writes, it is impossible to delegate the mmap() syscall to the server.

Fortunately if you build sqlite3 from source, you can disable the mmap() syscall by using specific compilation flags. After compiling sqlite3 from source with mmap() disabled I was able to get a list of syscalls I could intercept.[2]

This is the list of syscalls I had to implement in order to delegate all of sqlite3's execution to the server.

- access()
- close()
- fnctl()
- fdatasync()
- fstat()
- getcwd()
- ioctl()
- lseek()
- newfstatat()
- open()
- openat()
- pread()
- prlimit()
- pwrite()
- read()
- stat()
- unlink()
- write()

2.3 Client-server communication

There are many options when it comes to exposing the server API to the client. I've decided to use Unix domain sockets since they allow processes to communicate locally using the filesystem as their address space. This comes with the advantage of the possibility to easily switch to a network socket in the future.

The next step was to choose a communication protocol, at first I tried to implement my own protocol but I quickly abandoned the idea.
I have found a few viable options for the communication protocol: SunRPC + XDR, eRPC and gRPC + Protobuf.

	SunRPC + XDR	**eRPC**	**gRPC + Protobuf**
Primary use	Simple, classic C RPC	Embedded / real-time RPC	Modern service APIs
Dependencies	Low	Very low	High
Speed	Fast, low overhead	Very fast, minimal overhead	Moderate (higher overhead)
Transport	UDP, TCP	Any	HTTP/2
Adoption	Moderate (legacy, systems)	Niche (embedded)	Very high (cloud, microservices)

Table 2.1: Comparison of RPC frameworks

They all have their own advantages and disadvantages, I ultimately chose SunRPC + XDR[3] since it's lightweight, fast and widely used.

2.4 Server

The server implementations itself was quite straight forward. The main challenge was understanding how the syscalls work. I will be talking more about this in the chapter about the server implementation.

I've decided that the server will accept only one client connection, to allow me to progress faster on other parts of the project. It would have been nice to have the ability to server multiple clients, but I didn't have time to implement it. Nonetheless the current server architecture was more than enough for the scope of this project.

2.5 Notes

The research phase was heavily assisted by the professor, and also by AI.

This helped expedite the research process by providing a clearer direction on where and what to research.

Chapter 3

Implementation

3.1 Experimenting and use of AI

Since this project has many moving parts, the first thing I did was create small toy projects in order to better understand each tool and part of the project. I have created a few Unix socket client/server programs, and I've also played around with the syscall interception mechanism.

With the information gathered from building the client/server toy programs, I've used AI (Claude Code) to create the project's structure and boilerplate code. The code wasn't perfect, but it was a starting point.

The most value AI has brought to this project were the documentation files I've asked it to generate. I have explicitly asked the AI to create an extensive developer documentation on how all the project's pieces should function together. These documentation files reside in the *docs/dev_ docs/* folder.

This was especially useful for better understanding SunRPC + XDR, since I've found that online documentation is very hard to navigate.

3.2 Code structure

This project contains a lot of folders because I have decided to keep all code and documentation I wrote, as well as the sources for sqlite3. This is the project's directory structure:
`build/` - build artifacts.
`docs/` - markdown files for notes and documentation.

- `dev_docs/` - AI generated developer documentation
  - `00_README.md`
  - `01_INTRODUCTION.md`
  - `02_ARCHITECTURE.md`
  - ...
- `journal.md` - developer journal
- `sqlite3_syscalls.md` - list of syscalls used by sqlite3
- `todo.md` - tasks

`playground/` - experimental examples.

`sqlite-amalgamation-3510100/ and sqlite-src-3510100/` - sqlite3 source code.
`src/` - project's source code.

- `intercept/` - syscall interception headers
- `program.c` - test program
- `protocol/protocol.x` - protocol definition
- `rpc_client.c` - intercept client
- `rpc_server.c` - syscall server
- `transport_config.h` - transport layer config

`Makefile` - build scripts.
`sqlite3` - sqlite3 binary compiled without mmap().
`test_query.sql` - test SQL query.

3.2.1 playground/

The playground folder contains the tests programs written before starting the full implementation of the project

3.2.2 docs/

This folder contains markdown documents used to keep track of this project's todos, as well as developer documentation in the **dev_docs/** subfolder.
The files in the **dev_docs/** subfolder were generated by AI in the early stages of this project.

3.2.3 src/

This is the main source code for the project. It contains the client and server code, as well as the communication protocol definition.

3.3 Protocol

Let's start by analyzing the RPC protocol definition, since it's the glue between the client and the server.
The protocol definition lives in the *src/protocol/protocol.x* file.

```
/* Maximum path length for open() */
const MAX_PATH_LEN = 4096;

/* Maximum buffer size for read/write operations */
const MAX_BUFFER_SIZE = 1048576;  /* 1MB */

// Syscall request/response structures
struct open_request {
    string path<MAX_PATH_LEN>;  /* File path */
    int flags;                   /* Open flags (O_RDONLY, O_WRONLY, etc.) */
    unsigned int mode;           /* Permission mode */
};

struct open_response {
    int fd;          /* File descriptor (client-side mapped FD) */
    int result;      /* Return value: fd on success, -1 on error */
    int err;         /* errno value */
```

```
};

// ...

// RPC Program Definition
program SYSCALL_PROG {
    version SYSCALL_VERS {
        open_response SYSCALL_OPEN(open_request) = 1;
        openat_response SYSCALL_OPENAT(openat_request) = 2;

        // ...

    } = 1;  /* Version 1 */
} = 0x20000001;  /* Program number */
```

Listing 3.1: protocol.x code snippet

There are 3 main sections in the *protocol.x* file.

- **const declarations** - used as configuration variables
- **syscall request/response structs** - they define the data structures used for the client/server communication
- **RPC program definition** - defines the API exposed by the server by using the previously defined structs.

This file provides an abstract definition of our protocol. In order to make requests and receive responses to/from the server, we need access to the protocol's C implementation.

To generate the protocol's implementation we can run the following command:

Command Line

```
$ make rpc_gen

cd ./src/protocol/ && rpcgen -C protocol.x
```

This command generates 4 additional files in the *src/protocol/* folder:

- **protocol_clnt.c** - protocol client API.
- **protocol_svc.c** - protocol server API.
- **protocol_xdr.c** - serialization/deserialization logic.
- **protocol.h** - bundles request/response data structures, protocol configuration and client/server APIs.

3.4 Client

The client is implemented by the *src/rpc_client.c* file, while the system call override functions are located in the *src/intercept/* folder.
To compile the client you can use the following command:

Command Line

```
$ make client

gcc -Wall -g -I/usr/include/tirpc -I./src/protocol/ -I./src/
-shared -fPIC -o ./build/intercept.so \textbackslash ./src/rpc_client.c
\textbackslash ./src/protocol/protocol_xdr.c \textbackslash .
/src/protocol/protocol_clnt.c \textbackslash -ltirpc -ldl
```

The client is compiled as a shared library located in *build/intercept.so*, and it has 2 main functions:

- Establish connection to the server
- Intercept syscalls and forward them to the server

In order to use the client you must load the compiled shared library before all other shared libraries, by setting the *LD_PRELOAD=build/intercept.so*[4] environment variable.

Since our client shared library is loaded before all other libraries, including *libc* which contains the usual implementations of the system calls; we can override and, thus intercept, syscalls.

This is a simplified example of a syscall override:

```c
// Identical syscall signature as the one in 'libc'
int open(const char *pathname, int flags, ...) {

    CLIENT *client = get_rpc_client(); /* Get RPC client */
    int result = -1;

    if (client != NULL) {
        /* Prepare RPC request */
        open_request req;
        req.path = (char *)pathname; req.flags = flags; req.mode = mode;

        /* Call RPC service */
        open_response *res = syscall_open_1(&req, client);

        if (res != NULL) {
            /* RPC call succeeded */
            result = res->result;
            errno = res->err;
        } else {
            /* RPC call failed */
        }
    } else {
        /* No RPC connection - fall back to direct syscall */
    }

    return result;
}
```

Listing 3.2: simplified syscall override example

This is a simplified example, in the actual syscall implementation there are guards in place to prevent an interception loop, in the eventuality that a syscall interception function calls itself. Below you have a the complete decision tree that illustrates how the syscall interception works.

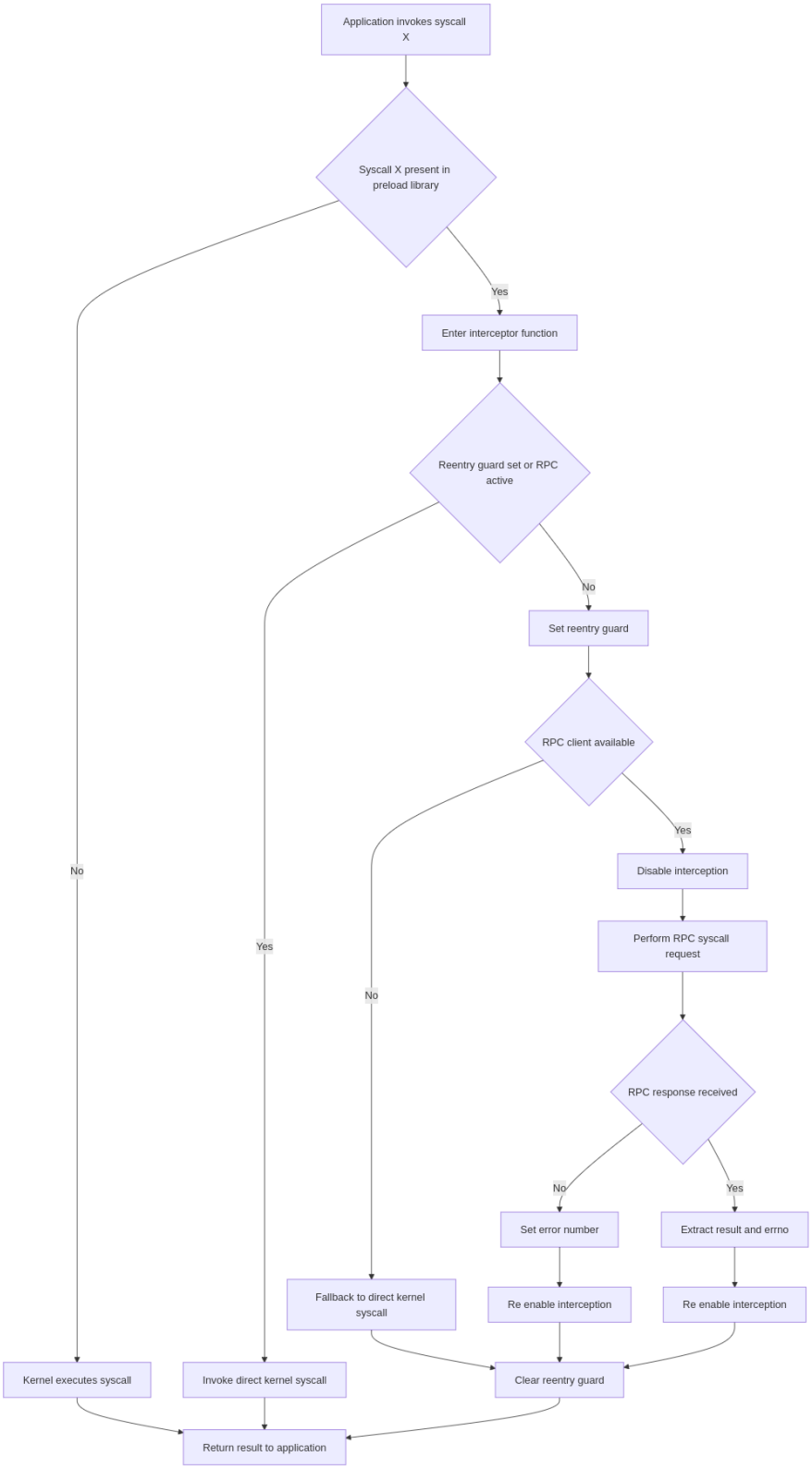

Figure 3.1: Syscall interception decision tree

3.5 Server

All the server's code is located in the *src/rpc_server.c* file and. In hindsight, it would have been a good idea to separate its logic into multiple files, like the client.

The server's responsibilities are:

- Listen for incoming connections
- Resolve remote syscall requests

For most syscalls, the server just executes them and returns their result to the client.

But for some syscalls there needs to be some extra logic in order to make them work. This is the case for all system calls that use file descriptors.

3.5.1 File descriptor mapping

Since file descriptors are assigned per process, the server receives the syscalls with the client file descriptors.
In order for the server to be able to use those file descriptors across syscalls, there is a mapping table put in place, that maps the client's file descriptor number to the server's file descriptor.

Below there's an example for an *open()*, *write()* and *close()* syscalls.

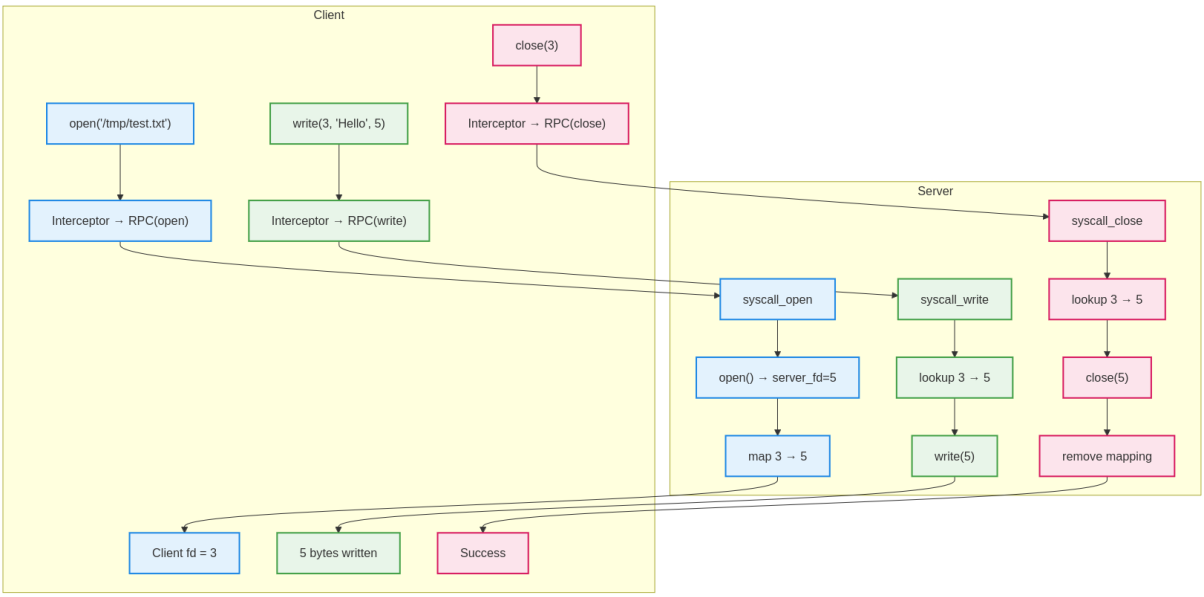

Figure 3.2: File descriptor mapping flow

The file descriptor mapping is done via an array called *fd_mapping*, and some functions that add, fetch and remove file descriptor mappings. Client file descriptors are used as indexes for the array.

3.6 How it all fits together

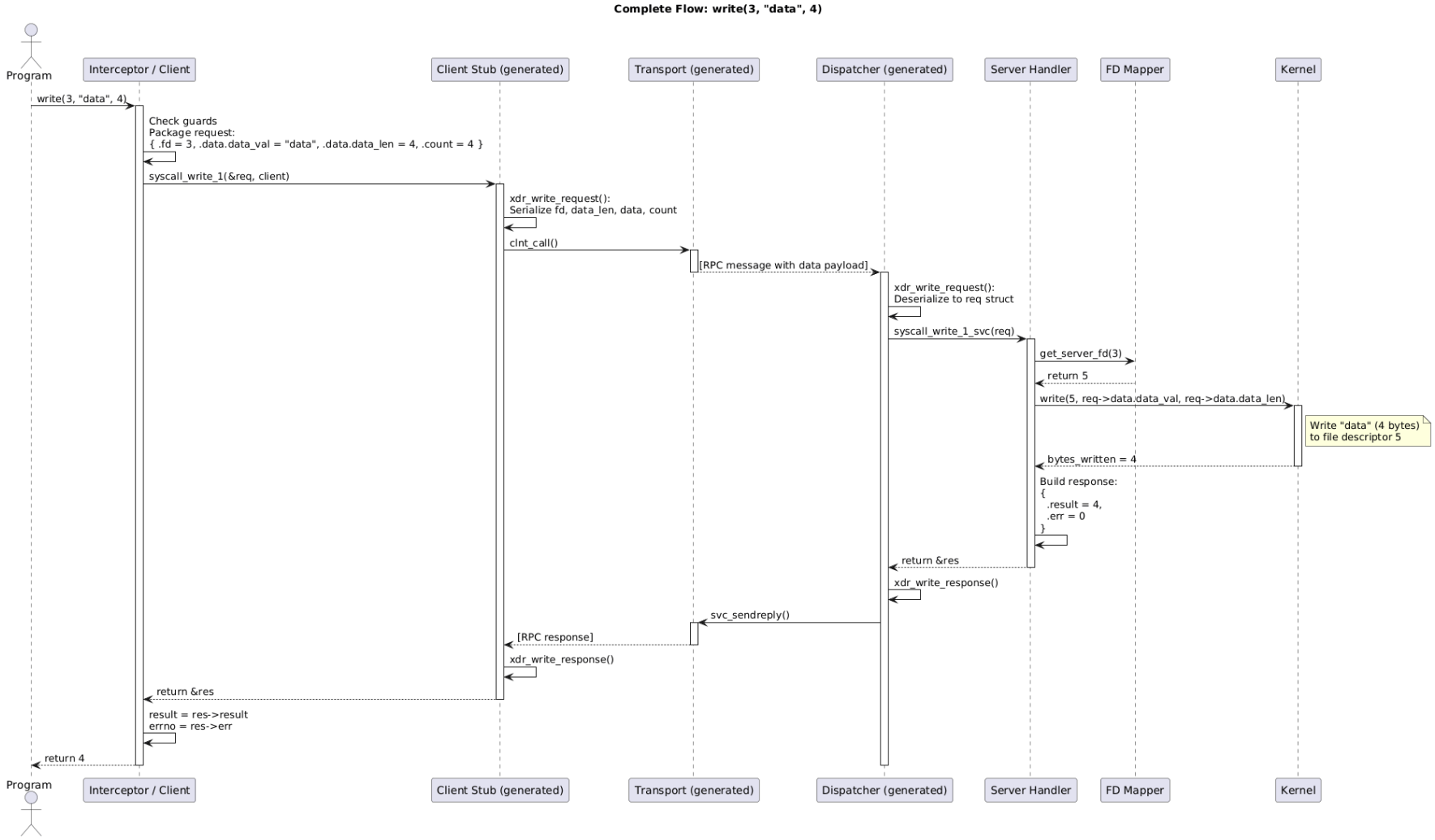

Figure 3.3: Complete app flow

3.7 What syscalls have been implemented

All the syscalls mentioned in the Research chapter have been implemented.

- access()
- close()
- fnctl()
- fdatasync()
- fstat()
- getcwd()
- ioctl()
- lseek()
- newfstatat()
- open()
- openat()
- pread()
- prlimit()
- pwrite()
- read()
- stat()
- unlink()
- write()

To get to this list I have run the *strace* command.

Command Line

```
$ make strace -o syscall_list.txt ./sqlite3 testdb.db
```

And within the sqlite3 CLI, I have run the following SQL queries:

```
1
2  CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,
     email TEXT UNIQUE NOT NULL);
3
4  CREATE TABLE orders (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT
     NULL, product TEXT NOT NULL, amount REAL NOT NULL, FOREIGN KEY (user_id)
     REFERENCES users(id));
5
6  INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com'), ('Bob',
     'bob@example.com');
7
8  INSERT INTO orders (user_id, product, amount) VALUES (1, 'Laptop', 1299.99),
     (1, 'Mouse', 25.50), (2, 'Keyboard', 75.00);
```

Listing 3.3: test_query.sql

Afterwards, I've used cyberchef.org to get a syscall list alongside their count. You can see the full strace output and the syscall list here.

Once I had the list, with help from the professor and the Linux manual pages, I chose the syscalls I had to implement.

I've implemented the syscalls in order, by their appearance count in the strace output file.

3.7.1 Syscall list and function

[5]

- access() - check user's permissions for a file
- close() - close a file descriptor
- fnctl() - manipulate file descriptor

- fdatasync() - synchronize a file's in-core state with storage device, so that all changed information can be retrieved even if the system crashes or is rebooted
- fstat() - get file status
- getcwd() - get current working directory
- ioctl() - control device, manipulates the underlying device parameters of special files. In particular, many operating characteristics of character special (e.g terminals)
- lseek() - reposition read/write file offset
- newfstatat() - get file status
- open() - open and possibly create a file
- openat() - open and possibly create a file
- pread() / pread64() - read from a file descriptor at a given offset
- prlimit() - get/set resource limits
- pwrite() / pwrite64() - write to a file descriptor at a given offset
- read() - read from a file descriptor
- stat() - get file status
- unlink() - delete a name and possibly the file it refers to
- write() - write to a file descriptor

As you can see, some of the syscalls do the same thing. This is because some are aliases and some do the same thing but with slightly different parameters.

In the interception code, I have tried to reuse as much code as possible. Here is an example:

```
1  ssize_t pread(int fd, void *buf, size_t count, off_t offset) {
2    // ... implementation here
3  }
4
5  // reuse pread()
6  ssize_t pread64(int fd, void *buf, size_t count, off_t offset) {
7    return pread(fd, buf, count, offset);
8  }
```

Listing 3.4: Alias function implementation in intercept_pread.h

3.8 Running and testing

In order to test and debug the client/server functionality, I first created a test program to test individual syscalls. Then I've used sqlite3.

The client and server print syscall interception/execution logs in the terminal via printf().

3.8.1 Individual syscalls

To test individual syscalls, I've created a test program: *src/program.c*.

The program has individual test functions for each syscall I've implemented that are ran sequentially in the main function. Here is an example:

```c
static int test_open_and_openat(void)
{
    printf("[open/openat] Testing open() and openat()\n");

    int fd = open(TEST_FILE, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0)
        return fail("open for write failed");

    printf("  open(): fd=%d\n", fd);
    close(fd);

    int fd_openat = openat(AT_FDCWD, TEST_FILE, O_CREAT | O_RDWR, 0644);
    if (fd_openat < 0)
        return fail("openat failed");

    const char *msg = "Testing openat syscall";
    if (write(fd_openat, msg, strlen(msg)) < 0)
        return fail("write after openat failed");

    close(fd_openat);
    printf("  openat(): success\n\n");
    return 0;
}
```

Listing 3.5: Test function example in program.c

3.8.1.1 Test program syscall interception output

Here you have what is displayed in the terminal when the *src/program.c* runs with syscall interception.

Strting the server:

Command Line

```
$  make run_server

./build/syscall_server
[Server] Starting RPC server...
[Server] Using UNIX transport
[Server] RPC server ready at /tmp/p3_tb
[Server] Waiting for connections...
```

Running the program.c with syscall interception also outputs the syscall interception client log messages:

Command Line

```
$  make run_program

LD_PRELOAD=./build/intercept.so ./build/program

=== Syscall Interception Test Program ===

[open/openat] Testing open() and openat()
[Client] Intercepted open("/tmp/p3_tb_test.txt", 577, 644)
[Client] open() RPC result: fd=3, errno=0
  open(): fd=3
[Client] Intercepted close(3)
[Client] close() RPC result: 0, errno=0
[Client] Intercepted openat(-100, "/tmp/p3_tb_test.txt", 66, 644)
[Client] openat() RPC result: fd=4, errno=0
[Client] Intercepted write(4, 0x56266ed5108b, 22)
[Client] write() RPC result: 22 bytes, errno=0
[Client] Intercepted close(4)
[Client] close() RPC result: 0, errno=0
  openat(): success

# ...

[Client] Intercepted unlink("/tmp/p3_tb_test.txt")
[Client] unlink() RPC result: 0, errno=0
=== Test Result: ALL TESTS PASSED ===
```

And the server terminal now displays:

Command Line

```
# ...

[Server] Accepted connection
[Server] Waiting for requests...
[Server] OPEN: path=/tmp/p3_tb_test.txt, flags=577, mode=644
[Server] FD mapping: client_fd=3 -> server_fd=3
[Server] OPEN result: fd=3, errno=0
[Server] CLOSE: client_fd=3
[Server] Removing FD mapping: client_fd=3 -> server_fd=3
[Server] CLOSE result: 0, errno=0

# ...
```

When the server is not available, the client intercepts uses direct syscalls.

Command Line

```
$ make run_program

LD_PRELOAD=./build/intercept.so ./build/program

=== Syscall Interception Test Program ===

[open/openat] Testing open() and openat()
[Client] Intercepted open("/tmp/p3_tb_test.txt", 577, 644)
[Client] Failed to connect to UNIX socket
[Client] No RPC connection, using direct syscall
  open(): fd=3
[Client] Intercepted close(3)
[Client] Failed to connect to UNIX socket
[Client] No RPC connection, using direct syscall

# ...
```

3.8.2 Running sqlite3

To run sqlite3, the manually compiled *sqlite3* binary must be used, because it has been compiled to not use mmap().

The syscalls are intercepted and forwarded to the server exactly like for the test program. Sqlite3 works "as intended", you can input queries in the CLI, while also seeing client outputs.

Command Line

```
$ LD_PRELOAD=./build/intercept.so ./sqlite3 test.db

[Client] Intercepted access("test.db", 0)
[Client] access() RPC result: -1, errno=2
[Client] Intercepted access("/home/catab/.config/sqlite3/sqliterc", 0)
[Client] access() RPC result: -1, errno=2
SQLite version 3.51.1 2025-11-28 17:28:25
Enter ".help" for usage hints.
[Client] Intercepted access("/home/catab/.local/state/sqlite_history", 0)
[Client] access() RPC result: -1, errno=2
sqlite> CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, ema
[Client] Intercepted access("test.db", 0)
[Client] access() RPC result: -1, errno=2
[Client] Intercepted getcwd(0x7ffee57a78e0, 4096)
[Client] getcwd() RPC result: "/home/catab/hearc/IL3/P3_TB", errno=0
[Client] Intercepted open("/home/catab/hearc/IL3/P3_TB/test.db", 655426, 644)

# ...
```

3.9 Known issues

There are two main issues with my implementation, one in the server and the other with sqlite3 interceptions.

3.9.1 Server socket

Since the server only accepts one client, it must be restarted each time you run the client.

This is caused because the connection reinitialization logic is not implemented. Once a client connects and then disconnects, the server is still listening to that 'ghost' connection and a new client instance cannot connect to it.

3.9.2 Sqlite3 interception

Even if the logs show all the interceptions, with no apparent issues, sqlite cannot properly format the database file when its syscalls are forwarded to the server.

Queries seem to run fine, but when trying to fetch data, for example, sqlite throws an error: *Runtime error: database disk image is malformed (11).*

I suspect this error is caused by an incorrect lseek syscall implementation on the server side. After some debugging, I still haven't been able to fix the issue.

Chapter 4

Results and benchmarks

This section presents the results produced by the implemented programs. Let's have a look at what was accomplished in this project, as well as the performance impact of the syscall interception logic.

4.1 What was accomplished

In this project I have successfully implemented a proof-of-concept of syscall interception server, as well as the client.

The research phase(s) of this project were the most valuable part of this project, since most of the topics covered by this project were unfamiliar to me.

I have learned a lot about the low-level Linux syscall API, RPC and C program debugging.

4.2 Benchmarks

Since system calls are a core functionality, it's very important to consider the performance impact of syscall delegation. I have performed some benchmarks to measure this impact.

4.2.1 Methodology

In order to get more accurate results I had to make sure that the measurements reflect only the impact of syscall delegation, with as little external influence as possible.

4.2.1.1 Minimizing external influences

I have closed all other programs on my computer, to avoid CPU usage fluctuations that may be caused by some other program starting to work on something.

4.2.1.2 Minimizing print statements

Since print statements are very expensive, I have overridden all printf() variants as well as any underlying non-syscall methods printf() might use.

This is accomplished by compiling and preloading another shared library. Its source code resides in *src/disable_print.c* and its shared library is *build/no_print.so*.

I decided to keep a few print statements in order to be able to see socket connection status and final test results. Those print statements are made by direct syscalls (cannot be intercepted), in combination with custom string concatenation methods.

4.2.1.3 Measurements

I have used the *time* command to time process execution.

In order to stress test my programs, I have run all the test functions in the program.c test program multiple times in a loop. Each iteration executes all the test functions once.

4.2.2 Performance impact

My implementation of the syscall delegation system is very performance taxing. Let's have a look at the numbers.

ITERATIONS	NO INTERCEPT	WITH INTERCEPT	DELTA	PERFORMANCE LOSS
1	0.2s	0.2s	0s	1x
100	0.02s	0.12s	0.1s	6x
1000	0.17s	0.92s	0.75s	5.41x
10000	1.59s	9.33s	7.74s	5.87x
100000	16.4s	91.07s	74.67s	5.55x

Table 4.1: Performance comparison with and without intercept.

4.2.3 Possible improvements

This is a very simple benchmark, it doesn't allow us to determine the cause of the 5x performance loss.

To fix the performance loss we would have to run the client and server through a profiler, in order to find out what specific operations slow down our programs.

Chapter 5

Conclusion

In conclusion, this project successfully demonstrated a proof-of-concept for a Linux syscall interception server and client. Through the development of this system, significant insights were gained into low-level Linux syscall APIs and RPC communication using SunRPC + XDR.

While the implementation intercepted and delegated 18 common syscalls, enabling the execution of basic test programs and a modified version of sqlite3, it also revealed a lot of challenges. Benchmarking indicated a substantial performance loss, with syscall delegation resulting in a performance loss of roughly 5.5x compared to direct execution. Furthermore, unresolved technical issues remain, such as the server's inability to handle multiple sequential client connections without a restart and a persistent data corruption error in sqlite3 likely linked to the lseek implementation.

List of Figures

Bibliography

[1] "What is confidential computing?" [Online]. Available: https://www.redhat.com/en/topics/security/what-is-confidential-computing

[2] "Compile-time Options." [Online]. Available: https://sqlite.org/compile.html

[3] "Sun RPC," Dec. 2025, page Version ID: 1329265558. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Sun_RPC&oldid=1329265558

[4] "Intercepting system calls with LD_preload - Sebastian Österlund." [Online]. Available: https://osterlund.xyz/posts/2018-03-12-interceptiong-functions-c.html

[5] "Linux man pages online." [Online]. Available: https://www.man7.org/linux/man-pages/